

Laboratorio di Programmazione

Lezione 6: Tipi di Dato

1	Qual è l'output?	2
2	Qual è l'errore?	3
3	Trova l'errore	4
4	Valore di default delle variabili	5
5	Operatori e precedenza	6
6	Divisione per zero	7
7	Overflow	8
8	Valori rappresentabili	9
9	Uguaglianza tra valori	10
10	Retta	11
11	Troncamento	13
12	Arrotondamento	14
13	Classifica triangolo	15

1 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var x int = 10
6     var y float64 = 2.5
7     t := 100
8     t, z, k := x/int(y), float64(x)/y, t*2
9     fmt.Println(t, z, k)
10 }
```

2 Qual è l'errore?

Di che tipo deve essere la variabile `c` perché la compilazione del codice non generi errori?

```
1 package main
2 import "fmt"
3
4 func main() {
5
6     var a, b int
7     fmt.Scan(&a, &b)
8
9     var c ???
10
11     c = a == b
12
13     if c {
14         fmt.Println("uguali")
15     } else {
16         fmt.Println("diversi")
17     }
18 }
```

3 Trova l'errore

Questo programma dovrebbe stampare il valore delle variabili a, b, c, d, ma contiene errori. Trovateli e correggeteli.

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a int = 4
6     var b float64 = 12.5
7     var c int
8     c := a + b
9
10    var d float64
11    d = a/c
12
13    fmt.Println(a, b, c, d)
14 }
```

4 Valore di default delle variabili

Che cosa succede se si prova a utilizzare una variabile senza averla inizializzata? Si esegua il seguente programma per stampare a video i valori di default (valori iniziali/zero-values) per i tipi `int`, `float64`, `string` e `bool`.

```
1 package main
2 import "fmt"
3
4 func main() {
5
6     var numeroIntero int
7     var numeroReale float64
8     var valoreLogico bool
9
10    fmt.Print("Valore di default per il tipo int: _", numeroIntero, "_\n")
11    fmt.Print("Valore di default per il tipo float64: _", numeroReale, "_\n")
12    fmt.Print("Valore di default per il tipo bool: _", valoreLogico, "_\n")
13
14 }
```

5 Operatori e precedenza

Cosa stampa questo frammento di codice?

```
1 var a, b, c int = 10, 5, 4
2 fmt.Println( a / b * c )
```

6 Divisione per zero

Cosa stampa questo frammento di codice?? Cosa stamperebbe se numero fosse di tipo int?

```
1 var numero float64 = 0
2 fmt.Println(numero)
3
4 fmt.Println(0 / numero)
5
6 numero = 1 / numero
7 fmt.Println(numero)
8
9 numero = 1 / numero
10 fmt.Println(numero)
11
12 numero = -(1 / numero)
13 fmt.Println(numero)
14
15 numero = numero - numero
16 fmt.Println(numero)
```

7 Overflow

Qual è l'output del seguente programma?

```
1 package main
2 import "fmt"
3
4 func main() {
5     var (
6         a, b int8 = 30, 100
7     )
8     somma := a + b
9     fmt.Println("La somma di", a, "e", b, "è", somma)
10 }
```

8 Valori rappresentabili

Il seguente programma stampa a video i limiti dei valori rappresentabili con i diversi tipi di dato numerici. Per ogni tipo di dato, i corrispondenti limiti sono definiti come costanti nel package `math`. Si stampi a video il valore di tali limiti eseguendo il programma.

```
1 package main
2
3 import (
4     "fmt"
5     "math"
6 )
7
8 func main() {
9     fmt.Println("uint8:", 0, math.MaxUint8)
10    fmt.Println("uint16:", 0, math.MaxUint16)
11    fmt.Println("uint32:", 0, math.MaxUint32)
12    fmt.Println("uint64:", 0, uint64(math.MaxUint64))
13
14    fmt.Println("int8:", math.MinInt8, math.MaxInt8)
15    fmt.Println("int16:", math.MinInt16, math.MaxInt16)
16    fmt.Println("int32:", math.MinInt32, math.MaxInt32)
17    fmt.Println("int64:", math.MinInt64, math.MaxInt64)
18
19    /* Floating-point limit values:
20    \begin{itemize}
21    \item Max is the largest finite value representable by the type.
22    \item SmallestNonzero is the smallest positive, non-zero value
23    \end{itemize}
24    representable by the type. */
25
26    fmt.Println("float32 - SmallestNonzero:", math.SmallestNonzeroFloat32)
27    fmt.Println("float32:", -math.MaxFloat32, math.MaxFloat32)
28    fmt.Println("float64 - SmallestNonzero:", math.SmallestNonzeroFloat64)
29    fmt.Println("float64:", -math.MaxFloat64, math.MaxFloat64)
30
31    fmt.Println("complex64:", complex(-math.MaxFloat32, -math.MaxFloat32),
32        complex(math.MaxFloat32, math.MaxFloat32))
33    fmt.Println("complex128:", complex(-math.MaxFloat64, -math.MaxFloat64),
34        complex(math.MaxFloat64, math.MaxFloat64))
35
36 }
```

9 Uguaglianza tra valori

Scrivere un programma che legge da standard input un numero reale x , ne calcola la radice quadrata r , usando `math.Sqrt`, e poi verifica se $x = r^2$. In caso affermativo stampa “ $x = r^2$ ”, altrimenti stampa “ $x \neq r^2$ ”, con i valori di x ed r sostituiti. Provate con $x = 9$ e $x = 10$ e spiegatevi il risultato.

Esempio d'esecuzione:

```
$ go run uguaglianza.go
Inserisci x: 4
4 = 2^2
```

10 Retta

Scrivere un programma che legge da standard input un intero s e due reali m, q , dove:

- s è il seme (seed) da usare per inizializzare il generatore di numeri casuali, usando `rand.Seed(s)`.
- m, q definiscono una retta $y = mx + q$ nel piano cartesiano.

Il programma poi calcola, per 10 volte, un punto casuale $p = (x, y)$. Per fare questo, genera sia x che y tramite `rand.Float64()`. Per ogni punto p generato il programma deve decidere se il punto sta sopra, sotto, o sulla retta, con una tolleranza di $\epsilon = 10^{-9}$. Il programma deve utilizzare almeno due tra le seguenti funzioni:

```
const EPSILON = 1.0e-9
```

```
func ÈMaggioreDiY(x, y float64) bool {  
    return (x - y) > EPSILON  
}
```

```
func ÈUgualeAY(x, y float64) bool {  
    return math.Abs(x - y) <= EPSILON  
}
```

```
func ÈMinoreDiY(x, y float64) bool {  
    return (x - y) < -EPSILON  
}
```

Esempio d'esecuzione:

```
$ go run retta.go  
Inserisci s: 2  
Inserisci m e q: 1 0
```

```
Punto (0.8364831721292811,1.325271527168901) - Il punto sta sopra la retta  
Punto (0.25744012651422055,0.5948870460174552) - Il punto sta sopra la retta  
Punto (3.071353082885974,4.638274793965569) - Il punto sta sopra la retta  
Punto (2.125002747927654,1.030771445518701) - Il punto sta sotto la retta  
Punto (1.059760133310769,0.6344848690006494) - Il punto sta sotto la retta  
Punto (3.09889044262683,4.311133779707338) - Il punto sta sopra la retta  
Punto (1.843213441859697,1.9013752063486216) - Il punto sta sopra la retta  
Punto (3.1086212867763456,4.552073840109743) - Il punto sta sopra la retta  
Punto (2.151010640152745,0.06284952382418317) - Il punto sta sotto la retta  
Punto (2.6797238953528506,4.4750914842551826) - Il punto sta sopra la retta
```

```
$ go run retta.go  
Inserisci s: 2  
Inserisci m e q: 1 1
```

```
Punto (0.8364831721292811,1.325271527168901) - Il punto sta sotto la retta  
Punto (0.25744012651422055,0.5948870460174552) - Il punto sta sotto la retta  
Punto (3.071353082885974,4.638274793965569) - Il punto sta sopra la retta
```

Punto (2.125002747927654,1.030771445518701) - Il punto sta sotto la retta
Punto (1.059760133310769,0.6344848690006494) - Il punto sta sotto la retta
Punto (3.09889044262683,4.311133779707338) - Il punto sta sopra la retta
Punto (1.843213441859697,1.9013752063486216) - Il punto sta sotto la retta
Punto (3.1086212867763456,4.552073840109743) - Il punto sta sopra la retta
Punto (2.151010640152745,0.06284952382418317) - Il punto sta sotto la retta
Punto (2.6797238953528506,4.4750914842551826) - Il punto sta sopra la retta

11 Troncamento

Scrivere un programma che legge da standard input un numero reale x e stampa il suo valore *troncato* alla seconda cifra decimale. A tale scopo manipolate opportunamente x tramite moltiplicazione e divisione, prestando attenzione ai tipi usati.

Esempio d'esecuzione:

```
$ go run troncamento.go
Inserisci il valore da troncare: 10.34762
Valore troncato = 10.34
```

```
$ go run troncamento.go
Inserisci il valore da troncare: 8.34267
Valore troncato = 8.34
```

Raffinate poi l'esercizio prendendo in input anche il numero n di cifre a cui troncare.

```
$ go run troncamento.go
Inserisci il valore da troncare e il numero di cifre: 10.34762 2
Valore troncato = 10.34
```

```
$ go run troncamento.go
Inserisci il valore da troncare e il numero di cifre: 10.34762 4
Valore troncato = 10.3476
```

12 Arrotondamento

Adattate l'esercizio 11 in modo da effettuare un *arrotondamento*.

Esempio d'esecuzione:

```
$ go run arrotondamento.go  
Inserisci il valore da arrotondare: 10.34762  
Valore arrotondato = 10.35
```

```
$ go run arrotondamento.go  
Inserisci il valore da arrotondare: 8.32467  
Valore arrotondato = 8.32
```

Raffinatelo per prendere in input anche il numero di cifre:

```
$ go run troncamento.go  
Inserisci il valore da arrotondare e il numero di cifre: 10.34762 2  
Valore troncato = 10.35
```

```
$ go run troncamento.go  
Inserisci il valore da arrotondare e il numero di cifre: 10.34762 4  
Valore troncato = 10.3476
```

13 Classifica triangolo

Scrivere un programma che legga da standard input tre coppie di reali (x_A, y_A) , (x_B, y_B) , (x_C, y_C) che identificano tre punti A, B, C nel piano cartesiano. Il programma deve poi decidere se il triangolo ABC è equilatero, isoscele o scaleno. La lunghezza di ciascun lato è la distanza euclidea tra i suoi estremi, per esempio:

$$|AB| = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

Due lunghezze vanno considerate uguali se differiscono di al più $\text{EPSILON} = 1.0\text{e-}9$; si vedano gli esercizi precedenti. Inoltre, il programma deve definire e usare una funzione

Distanza(x1, y1, x2, y2 float64) float64

che calcola la distanza euclidea tra due punti (x_1, y_1) e (x_2, y_2) .

Esempio d'esecuzione:

```
$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: -6 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: -1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto C: -3.5 5.330127018922193
```

Il triangolo ABC è equilatero.
Lunghezza del lato: 5

```
$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: -6 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: -1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto C: -3.5 5.33
```

Il triangolo ABC è isoscele.
I lati di lunghezza uguale sono AC e BC.
Lunghezza dei lati AC e BC: 4.999889998789973
Lunghezza del lato AB: 5

```
$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: 1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: 3 3
Inserisci i valori dell'ascissa e dell'ordinata del punto C: 1 5
```

Il triangolo ABC è isoscele.
I lati di lunghezza uguale sono AB e BC.
Lunghezza dei lati AB e BC: 2.8284271247461903
Lunghezza del lato AC: 4

```
$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: 1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: 1 5
Inserisci i valori dell'ascissa e dell'ordinata del punto C: 5 1
```

Il triangolo ABC è isoscele.

I lati di lunghezza uguale sono AB e AC.
Lunghezza dei lati AB e AC: 4
Lunghezza del lato BC: 5.656854249492381

\$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: 1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: 1 5
Inserisci i valori dell'ascissa e dell'ordinata del punto C: 3 3

Il triangolo ABC è isocele.
I lati di lunghezza uguale sono AC e BC.
Lunghezza dei lati AC e BC: 2.8284271247461903
Lunghezza del lato AB: 4

\$ go run classifica.go
Inserisci i valori dell'ascissa e dell'ordinata del punto A: 1 1
Inserisci i valori dell'ascissa e dell'ordinata del punto B: 1 4
Inserisci i valori dell'ascissa e dell'ordinata del punto C: 1.5 1.5

Il triangolo ABC è scaleno.
Lunghezza del lato AB: 3
Lunghezza del lato BC: 2.5495097567963922
Lunghezza del lato AC: 0.7071067811865476